Unit - 1

Question:
How do program organizations differ in structured programming and object-oriented programming? Also, distinguish between Procedure Oriented Programming and Object Oriented Programming.

Answer:

Programs can be organized in two general ways:

- **Around code**: Structured programming method (e.g., in C).
- **Around data**: Object-oriented programming method (e.g., in C++).

- **Structured Programming:**
  - Focus is on *what is happening* (functions and procedures).
  - Functions act on data.
  - Any function can operate on any type of data in the program.

- **Object-Oriented Programming:**
  - Focus is on *who is being affected* (data).
  - Data controls access to functions.
  - Only routines (functions) permitted by the data type can act on that data.

Difference Between Procedure Oriented Programming and Object Oriented Programming:

Procedure Oriented Programming	Object Oriented Programming
Program is divided into functions	Program is divided into objects
Follows top-down approach	Follows bottom-up approach
No access specifiers	Uses access specifiers like private, public, etc.
Data is less secure	Supports data hiding (more secure)
Adding new data/functions is hard	Adding new data/functions is easy
No concept of overloading	Supports overloading
Inheritance and data hiding not supported	Supports inheritance and data hiding
Functions are more important than data	Data is more important than functions
Based on unreal world modeling	Models real-world entities

Suitable for medium-sized programs	Suitable for large and complex programs
Uses procedure abstraction	Uses data abstraction
No code reusability	Promotes code reusability
Examples: C, FORTRAN, Pascal	Examples: C++, Java, Python

Question:
Explain the concept of Encapsulation in C++ and how it relates to Data Encapsulation.

Answer:

- Encapsulation:
    - It is the mechanism that binds together code and data it manipulates.
    - Prevents outside interference and misuse of data.
    - When code and data are combined in a way that forms a self-contained unit, an object is created.
    - This object functions as a "black box"—internally complex but externally simple to use.
    - Inside an object:
        - Private members: Accessible only by the object itself.
        - Public members: Provide controlled access to internal private elements.
    - Encapsulation ensures a clear boundary between internal implementation and external use.
    - An object in OOP is essentially a compound variable that encapsulates data and functions.

- Data Encapsulation:
    - Combines data and functions into a single unit called a class.
    - Direct access to data is restricted; access is only via functions inside the class.
    - Enables data hiding, where internal object details are concealed from outside.

Question:
Explain the concept of Polymorphism in C++ with reference to its meaning, usage, and types. Also mention how overloading relates to polymorphism.

Answer:

It allows one interface to control access to a general class of actions.

Specific action selected depends on the situation/context.

<u>Programming Example:</u>

    - You can define three types of stacks:
        - Stack for integers
        - Stack for characters
        - Stack for floating-point values
    - Use the same function names: `push()` and `pop()` for all.
    - Compiler chooses the correct version based on data type.

<u>Advantages:</u>

    - Reduces complexity by enabling the same interface to access multiple actions.
    - Programmer uses only the general interface; compiler handles method selection.

<u>Types in C++:</u>

    - Compile-time Polymorphism: Achieved through function and operator overloading.
    - Run-time Polymorphism: Achieved through virtual functions and inheritance.

<u>Overloading (as a form of Polymorphism):</u>

- Overloading is a type of polymorphism.
- Allows the same operator or function to behave differently depending on context.
- Occurs when an existing operator or function is made to work with new data types or classes.
- Enables multiple implementations under a single name.

Question:
Explain the concept of Inheritance in C++ and state its significance in Object-Oriented Programming.

Answer:

    - Inheritance is the process of forming a new class from an existing class (base class).
    - The existing class is called the base class (also: parent class or super class).
    - The new class formed is called the derived class (also: child class or sub class).

- Functionality:
    - A derived class acquires properties (data and functions) of the base class.
    - Inheritance supports the concept of classification and hierarchical structure.

- Significance:
    - Enables code reuse by avoiding redefinition of shared characteristics.
    - Helps in managing knowledge and behavior through hierarchical classification.
    - Reduces overall code size and improves structure.

    - Facilitates the creation of specific instances of general classes.

Question:
What is Data Abstraction in C++ and how does it enhance programming?

Answer:

- Data Abstraction increases the power of the programming language by enabling the creation of user-defined data types.
- It allows a program to represent only the necessary information, while hiding the implementation details.
- Through abstraction, users interact with an interface without needing to understand the internal workings.
- Helps in simplifying complex systems by exposing only relevant features.

Question:
Explain the usage and behavior of the I/O operators (<< and >>) in C++. Mention any limitations with an example.

Answer:

The << and >> operators are used for output and input, respectively.

They can handle all of C++'s built-in data types.
Multiple inputs/outputs can be chained in a single statement.

<u>Example Program:</u>

```
#include <iostream>

using namespace std;

int main()

{

    float f;

    char str[80];

    double d;

    cout << "Enter two floating point numbers: ";

    cin >> f >> d;

    cout << "Enter a string: ";

    cin >> str;

    cout << f << " " << d << " " << str;

    return 0;

}
```

Behavior:

- When entering a string like: `This is a test`, only **"This"** is read.
- This is because the `>>` operator **stops reading at the first whitespace** character.
- `"is a test"` is not captured due to this limitation.

Question:
How are local variables declared in C++? What are the benefits and considerations of declaring them at the point of first use?

Answer:

In C++, local variables can be declared at any point within a block, not just at the beginning.

This allows variables to be declared close to where they are first used.

Benefits:

- Improves encapsulation by keeping code and data closely related.
- Helps avoid accidental side effects by reducing the scope of variables.
- Especially useful in large or complex functions, where it enhances clarity.
- Supports writing more readable and maintainable code by limiting variable visibility.

Considerations:

- In short functions, declaring variables at the start may still be simpler and more consistent.
- Some programmers prefer all declarations at the top for easier visibility of used variables.
- Decision should be based on the size and complexity of the function.

Question:
What are the rules and features of variable declarations and initializations in C++? Include behavior in `for` loops, global variables, and arrays.

Answer:

Variable Declarations in C++:

- Variables are **not restricted to the start of a block**.
- You can **interleave declarations and statements** as needed.
- Declaring variables at the beginning is still **considered good style**.

Example:

```
void main() {
    int I = 5;
    printf("Please enter J: ");
    int J;  // Declared after a statement — allowed in C++
    scanf("%d", &J);
}
```

Counter Variables in a `for` Loop:
C++ allows declaring the loop variable within the `for` statement.
The variable exists only inside the loop and is destroyed after the loop ends.

Example:

```
#include <iostream>
using namespace std;

int main() {
    for (int I = 0; I < 5; I++)
        cout << I << endl;
    return 0;
}
```

Initializing Global Variables:
Global variables can be initialized using any valid expression, not just constants.

Example:

```
int rows = 5;

int cols = 6;

int size = rows * cols;  // Valid global initialization
```

Initializing Array Elements:
C++ allows initializing arrays using expressions, including pointers or addresses.

Example:

```
void main() {

    int n1, n2, n3;

    int *nptr[] = { &n1, &n2, &n3 };  // Array of pointers
initialized with addresses

}
```

Question:
What is the status of the "default-to-int" rule in standard C++?

Answer:

In standard C++, the "default-to-int" rule is removed.
If no explicit data type is specified in a declaration, the compiler will generate an error.
The rule previously allowed functions or variables to be declared without a type, assuming `int` by default.

Example of invalid code in standard C++:

```
func(int i)  // Error: missing return type
     {
         return i * i;
     }
```

Correct usage in standard C++:

```
int func(int i)
{
    return i * i;
}
```

All data types must be explicitly declared in modern C++.
This ensures clarity, consistency, and compliance with the language standard.
Do not use default-to-int in new C++ code.

Question:
What is the `bool` data type in C++? How does it behave and interact with other types and operators?

Answer:

- C++ introduces a built-in Boolean data type called `bool`.
- Objects of type `bool` can store only two values: `true` and `false`.
- These are keywords defined in the language:
  - `true` is internally represented as 1
  - `false` is internally represented as 0
- C has no explicit type for true/false values.

<u>Conversions:</u>

- C++ allows implicit conversions between `bool` and other integral types:
  - Any non-zero value is converted to `true`.
  - A zero value is converted to `false`.
  - `true` is converted to integer 1, and `false` to integer 0.

<u>Usage with Operators:</u>

- Operators that both require and produce `bool` values:
  - Logical operators: `&&` (AND), `||` (OR), `!` (NOT)
- Relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) always return a `bool` result.

<u>Other integral types (int, char, etc.) are implicitly converted to bool when appropriate:</u>

- Non-zero values are converted to `true`.
- Zero values are converted to `false`.

<u>Usage in Control Statements:</u>

- Boolean expressions are used in conditional and loop constructs:

- `if (boolean_expression)`
- `while (boolean_expression)`
- `do { } while (boolean_expression)`
- `for ( ; boolean_expression; )`

`bool` provides a clear and explicit way to handle true/false logic in C++.
It enhances code readability and enforces logical correctness in expressions and conditions.

Question:
What are the new-style C++ headers? How do they differ from the old-style C or C++ headers, and what is the current recommendation for their use?

Answer:

In C++, when using a library function, its header must be included using the `#include` statement.
Example from C:
`#include <stdio.h>` includes the I/O function header `stdio.h`.

<u>Old-style C++ headers:</u>

- When C++ was first invented, it used the same C-style headers.
- These are header files with **`.h`** extensions.
- Example:
  `#include <iostream.h>` includes the file `iostream.h`.

<u>New-style C++ headers (Standard C++):</u>

- These headers do not specify filenames.
- They are abstractions that guarantee appropriate prototypes and definitions are declared.
- They are included without the **`.h`** extension, using just the header name in angle brackets.
- Examples:
  - `<iostream>`
  - `<fstream>`
  - `<vector>`
  - `<string>`

Included using the same `#include` syntax:
#include <iostream>

<u>C standard headers in C++:</u>

- C++ includes the entire C function library and still supports C-style headers like `stdio.h`, `ctype.h`.
- Standard C++ defines new-style replacements for these:

- Add a `"c"` prefix and remove `.h`.
- Examples:
  - `math.h` → `<cmath>`
  - `string.h` → `<cstring>`

Recommendation and Compatibility:

- New-style headers are preferred.
- Including C-style headers is still permissible but is deprecated by Standard C++.
- As of now, all C++ compilers support old-style headers, but their use in new programs is not recommended.
- Old-style headers are still common in older programs, but they are declared obsolete.

Question:
What is the use of `void*` in C and C++? Explain with pointer casting rules and an example.

Answer:

In C, it is legal to cast any pointer to or from a `void*` implicitly.
In C++, implicit casting to or from `void*` is not allowed. You must use an explicit cast.
This is because C++ enforces stronger type safety than C.

Example:

```
int N;

int *P = &N;

void *Q = P;              // illegal in C++ (implicit cast)

void *R = (void *) P;     // legal in C++ (explicit cast)
```

Explanation:

- `Q = P;` will give a compiler error in C++ because it implicitly converts `int*` to `void*`.
- `R = (void*)P;` is valid in C++ because the cast is explicit, telling the compiler that the conversion is intentional.

Miscellaneous Topics:

NULL in C++

- C++ does not use the value NULL, instead NULL is always 0 in C++, so we simply use 0
- Example:

```
int *P = 0; // equivalent to
            // setting P to NULL
```

- Can check for a 0 pointer as if true/false:

```
if (!P)  // P is 0 (NULL)
  ...
else // P is not 0 (non-NULL)
  ...
```

Read C++ Unit - 1 pdf Pages 49 - 80, 108, 111, 146-165

Question:
What is a `struct` in C++? How are tags used with `struct` declarations?

Answer:

A `struct` (short for structure) in C++ is a user-defined data type that allows grouping of variables of different types into a single unit.
Structures are commonly used to represent real-world entities (like a student, point, etc.) that have multiple attributes.

<u>Structure Declaration:</u>

```
struct MyType {
    int A;
    float B;
};
```

- `MyType` is the tag name (also called type name) of the structure.
- `A` and `B` are data members of the structure.

<u>Creating Structure Variables:</u>

```
MyType V;
```

- In C++, you can use the tag name directly to declare variables — no need to write struct MyType V; (which is required in C).
- This is because in C++, the struct tag also acts as a type name.

<u>Explanation of Tags:</u>

- A tag is the name given to a struct, union, or enum to refer to the data type later.
- In C++, once the structure is defined with a tag (like MyType), you can use it as a data type directly, just like int or float.

Question:
What is an `enum` in C++? How does it differ from C in terms of usage?

Answer:

An enum (enumeration) in C++ is a user-defined type consisting of a set of named constant values.
It is used to represent integral constants in a readable and organized way.

Basic Syntax:

```
enum Color { red, blue, green };
```

- Color is the enum type.

- red, blue, and green are enumerator constants that are implicitly assigned integer values starting from 0.

- In C, enums are treated as integers, so assigning an integer like 1 or doing ++c is allowed.
- In C++, enums are strongly typed and not implicitly treated as integers.
    - This means operations like assigning an int to an enum or incrementing an enum variable directly are not allowed.
    - These restrictions enforce type safety in C++.

Question:
What are namespaces in C++ and why is using namespace std; commonly used?

Answer:

Namespaces are used to organize code into logical groups and to prevent name collisions, especially in programs using multiple libraries.

A namespace is a declarative region that provides a scope for identifiers (such as variables, functions, classes).

All identifiers at namespace scope are visible to one another without qualification.

For compatibility, when a C++ program includes a C header like `stdio.h`, its contents remain in the global namespace.
This allows C++ compilers to compile C-subset programs.

<u>Purpose and Behavior:</u>

- Elements declared in one namespace are separate from those in another.
- Originally, C++ library components were placed in the global namespace (like in C).
- With the introduction of new-style headers, their contents were placed in the `std` namespace to avoid name conflicts.

The `std` Namespace:

- The `std` namespace contains many standard library components, including:
  - Data types
  - Functions
  - Objects

When you write: using namespace std;

  - You're telling the compiler to treat all names in the `std` namespace as if they're in the global namespace.
  - This makes it unnecessary to qualify names with `std::` (e.g., `cout`, `cin`, etc.)

Example:

Without `using namespace std;`:

```
std::cout << "Hello";
```

With `using namespace std;`:

```
cout << "Hello";
```

Question:
What is a class in C++? Explain its structure, key components, and how it supports object creation and encapsulation. Also explain how member functions are defined and accessed.

Answer:

A class is C++'s most important feature. It is used to define a new user-defined data type.
A class defines the general form of an object.
The keyword `class` is used to declare a class. Variables have values for fields of the class.
Variables created of a particular class are instances of that class

Syntax:

```
Class class_name {

............. ............... .............

}
```

Key Concepts:

A class may contain private and public parts.
By default, all members of a class are private.
Items after the `public:` keyword are public and accessible from outside the class.
Use of private members supports encapsulation, by restricting access to internal details.

Terminology:

- Member functions: Functions defined inside a class.
- Member variables (data members): Variables declared inside a class.

Object Creation:

Once a class is defined, **objects** of that class can be created.

```
Circle c;       // creates an object

Circle *cptr;   // pointer to a Circle object
```

Example:

```
stack mystack;
```

An object is an instance of a class.
A class is a logical abstraction, whereas an object is real and exists in memory.

General Form of a Class Declaration:

```
class class-name {
  private data and functions
public:
  public data and functions
} object name list; // optional
```

Function Definitions:

- All functions in C++ must be prototyped.
- When defining a member function outside the class, use the scope resolution operator `::`

Example:

```
void stack::push(int i) {
  if(tos == SIZE) {
    cout << "Stack is full.\n";
    return;
  }
  stck[tos] = i;
  tos++;
}
```

The `::` **operator** tells the compiler which class the function belongs to.

Accessing Members:

To access a member function or data member **from outside the class**, syntax:

```
object_name.member
```

Access Control:

Private members of a class are accessible only by functions that are members of that class.

Write a long code using classes

Question:
Differentiate between structures and classes in C++.

Answer:

Structures and classes in C++ are similar, but they differ mainly in default access specifier and intended use. Both support functions, constructors, destructors, and access specifiers in C++.

Differences Between Structures and Classes in C++:

Feature	Structure (`struct`)	Class (`class`)
Default Access Specifier	`public`	`private`
Intended Use	Traditionally for passive data structures	Used for object-oriented programming
Data Hiding	Members are public by default → less secure	Members are private by default → better encapsulation
Inheritance Access Type	Inherited as `public` by default	Inherited as `private` by default
Function Members	Supported	Supported

Use in OOP	Rarely used for full object-oriented design	Designed for full OOP implementation
Syntax	`struct MyStruct { ... };`	`class MyClass { ... };`

Question:
What are constructors and destructors in C++? Explain their purpose, syntax, and behavior.

Answer:

<u>Constructors:</u>

- A constructor is a **special member function** of a class.
- It is **automatically called** when an object is created (instantiated).
- The constructor has the **same name** as the class.
- It has **no return type**, not even `void`.
- It is used to **initialize data members** of the class.
- A constructor can be **overloaded** — i.e., multiple constructors with the same name but different parameter lists.
- A constructor that takes **no parameters** is called the **default constructor**.
- Constructors may also have **default arguments**.
- They are typically declared in the **public section** of a class.
- They **allocate space** for the object and ensure it is **initialised correctly**.
- A constructor cannot have its **address taken**.
- They may make **implicit calls to the operator `new`**.

Example:

```
#include <iostream>
using namespace std;

class Hello {
public:
    Hello();  // constructor
};

// constructor definition
Hello::Hello() {
    cout << "Hello, Constructor!" << endl;
}

int main() {
```

```
        Hello obj;  // constructor is called automatically
        return 0;
    }
```

Constructor Behavior:

- For local objects, the constructor is called each time the object is declared.
- For global or static local objects, it is called once at initialization.
- A constructor executes automatically when the object is created.

Destructors:

- It is a member function which deletes an object.
- A destructor function is called automatically when the object goes out of scope:
    1. the function ends
    2. the program ends
    3. a block containing temporary variables ends
    4. a delete operator is called
- It has the same name as the constructor, but is preceded by a tilde ~.
- It has no return type and no arguments.
- Used to deallocate dynamic memory and release system resources acquired during the object's lifetime.
- Local objects are destroyed when their block ends.
- Global objects are destroyed when the program terminates.
- The destructor handles cleanup operations when an object goes out of scope.
- If the destructor is not defined, the compiler generates a default destructor.
- When a class contains a pointer to memory you allocate, it is your responsibility to release the memory before the class instance is destroyed.

Example:

```
INPUT:
#include <iostream>
using namespace std;

class Hello {
public:
    Hello();   // constructor
    ~Hello();  // destructor
};

Hello::Hello() {
    cout << "Constructor called\n";
}

Hello::~Hello() {
    cout << "Destructor called\n";
}

int main() {
```

```
        Hello obj;  // constructor is called here
        return 0;   // destructor is called here (when obj goes
    out of scope)
    }

    OUTPUT:
    Constructor called
    Destructor called
```

Question:
What is the default behavior of constructors in C++ if no constructor is explicitly defined?

Answer:

- If no default constructor is defined, then when an object is created, the compiler will only allocate memory (store) for its member objects.

- If the member objects themselves have default constructors, then those constructors will be called automatically to initialize the member objects.

- When constructors are explicitly declared in a class, then initialization of class objects becomes mandatory—you must provide the required arguments when creating the object.

Question:
What are parameterized constructors, copy constructors, and constructors with default arguments in C++? Explain with rules and examples.

Answer:

<u>Parameterized Constructor:</u>

- Used when different objects need to be initialized with different values at creation.
- A constructor that takes arguments is called a parameterized constructor.
- Syntax allows arguments to be passed directly during object creation.

Example:

```
 #include <iostream>
using namespace std;

class myclass {
    int a, b;
public:
    myclass(int i, int j) {
        a = i;
        b = j;
    }
```

```
    void show() {
        cout << a << " " << b;
    }
};

int main() {
    myclass ob(3, 5);  // or: myclass ob = myclass(3, 5);
    ob.show();
    return 0;
}
```

Additional Rules for Parameterized Constructors:
Constructor functions can be defined as inline.
Parameters of a constructor can be of any type except the class type itself.

Example of invalid constructor:

```
class myclass {

    myclass(myclass);  // illegal
};
```

Copy Constructor:
A constructor that accepts a reference to its own class type as a parameter.

It is called when:

- A new object is initialized from an existing object.
- An object is passed by value.
- An object is returned by value from a function.

It is not called during assignment after an object is already created.

Syntax:

```
    class myclass {
        myclass(myclass &);  // valid copy constructor
    };
```

Example:

```
    class Point {
    public:
        double x, y;
        Point() { x = y = 100; }
        Point(Point &p) {
            x = p.x;
            y = p.y;
        }
    };
    int main() {
        Point a;          // default constructor
        Point b(a);       // copy constructor called
```

```
    Point c = a;     // copy constructor called
    Point d;
    d = a;           // copy constructor NOT called
    return 0;
}
```

Example with Parameterized and Default Constructor:

```
INPUT CODE:
#include <iostream>
using namespace std;

class CRectangle {
    int width, height;
public:
    CRectangle();               // default constructor
    CRectangle(int, int);       // parameterized constructor
    int area() { return width * height; }
};

CRectangle::CRectangle() {
    width = 5;
    height = 5;
}

CRectangle::CRectangle(int a, int b) {
    width = a;
    height = b;
}

int main() {
    CRectangle rect(3, 4);    // uses parameterized
constructor
    CRectangle rectb;         // uses default constructor

    cout << "rect area: " << rect.area() << endl;
    cout << "rectb area: " << rectb.area() << endl;
    return 0;
}
Output:
rect area: 12
rectb area: 25
```

Constructors with Default Arguments:

Formal parameters are given default values to use when actual parameters are missing.
Actual values, if provided, will override the default ones.
If called with no arguments, it acts like a default constructor.

Example:

```
class X {
public:
    X(int arg1 = 7, int arg2 = 9);
};

X::X(int arg1, int arg2) {
    // constructor body
}
```

Question:
What is dynamic initialization of objects in C++? Explain with the given example.

Answer:

- Dynamic initialization refers to initializing an object using values provided at runtime, typically through user input or calculations.
- This is achieved by calling a parameterized constructor with dynamic values.
- A dynamic constructor is a constructor that allocates memory at runtime using the `new` operator.
- It is typically used when the size of a data member (like a character array) is not known at compile time and must be determined during execution.

Example:

```
INPUT:
#include <iostream>
#include <cstring>
using namespace std;

class Person {
    char *name;  // dynamic
    int age;     // static
    int id;      // static

public:
    // Constructor with parameters (dynamic + static
initialization)
    Person(const char *n, int a, int i) {
        int len = strlen(n);
        name = new char[len + 1];    // dynamic allocation
        strcpy(name, n);             // copy input name
        age = a;
        id = i;
    }

    // Display function
    void show() {
```

```
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "ID: " << id << endl;
    }

    // Destructor to free memory
    ~Person() {
        delete[] name;
    }
};

int main() {
    char tempName[50];
    int tempAge, tempID;

    cout << "Enter name: ";
    cin >> tempName;

    cout << "Enter age: ";
    cin >> tempAge;

    cout << "Enter ID: ";
    cin >> tempID;

    Person p(tempName, tempAge, tempID);  // dynamic +
parameterized constructor
    cout << "\nPerson details:\n";
    p.show();

    return 0;
}
OUTPUT:
Person details:
Name: Rahul
Age: 21
ID: 1001
```

Question:
How are comments written in C++? Explain both types with examples.

Answer:

Comments in C++

- Can use C form of comments /* A Comment */
- Can also use // form:
  - when // encountered, remainder of line ignored
  - works only on that line
- Examples:

```
void main() {
  int I;  // Variable used in loops
  char C; // No comment comment
```

CATEGORIES OF OPERATORS IN C++ (Based strictly on the PDF content)

C++ provides several types of operators, grouped based on the kind of operations they perform.

1. Arithmetic Operators

- Perform basic mathematical operations.
- Examples: `+`, `-`, `*`, `/`, `%`

Example:

```
int a = 10, b = 3;
cout << a + b;  // Output: 13
cout << a % b;  // Output: 1
```

2. Relational Operators

- Compare two values and return a `bool` (true/false).
- Examples: `==`, `!=`, `<`, `>`, `<=`, `>=`

Example:

```
int a = 10, b = 3;
cout << (a > b);  // Output: 1 (true)
cout << (a == b); // Output: 0 (false)
```

3. Logical Operators

- Used for combining multiple boolean expressions.
- Examples: `&&` (AND), `||` (OR), `!` (NOT)

Example:

```
bool x = true, y = false;
cout << (x && y);  // Output: 0
cout << (x || y);  // Output: 1
```

4. Assignment Operators

- Used to assign values to variables.
- Examples: `=`, `+=`, `-=`, `*=`, `/=`, `%=`

Example:

```
int x = 5;
x += 3;  // x becomes 8
```

5. Increment and Decrement Operators

- Increase or decrease a variable by 1.
- Examples: `++`, `--`
- Can be used as prefix or postfix.

Example:

```
int x = 5;
cout << ++x; // Output: 6
cout << x--; // Output: 6, then x becomes 5
```

6. Conditional (Ternary) Operator

- Short form of `if-else`.
- Syntax: `condition ? expression1 : expression2`

Example:

```
int a = 10, b = 5;
int max = (a > b) ? a : b;  // max = 10
```

7. Bitwise Operators

- Perform operations on bits.
- Examples: `&`, `|`, `^`, `~`, `<<`, `>>`

Example:

```
int a = 5, b = 3;
cout << (a & b);  // Output: 1 (0101 & 0011 = 0001)
```

8. Scope Resolution Operator (`::`)

- Used to define a class's member function outside the class.

Example:

```
class Test {
  public:
    void show();
};

void Test::show() {
  cout << "Hello";
}
```

9. Member Access Operators

- Dot (`.`): Used to access members using object.
- Arrow (`->`): Used to access members using pointer.

Example:

```
object.memberFunction();     // Using .
pointer->memberFunction();   // Using ->
```

10. Input/Output Operators

- `<<` : Insertion operator (used with `cout`)
- `>>` : Extraction operator (used with `cin`)

Example:

```
int x;
cin >> x;
cout << "Value: " << x;
```